NERDS MASTERMIND · FIELD NOTES

The Anti-Sprawl Playbook

Your AI agents keep breaking, burning money, and drifting off-track? Here's **why** — and the exact order to fix it. No code background needed.

Built for **Michael** & the crew after the 6/19 session · curated by **Chris Lamm**
⏱ 4-min skim · 🛠 real tools + commands · 🚦 tagged for every skill level. Start at the TL;DR, go as deep as you want.

The 60-second version

If you remember nothing else from the room:

- **Sprawl & drift are the disease.** Too many half-built workflows + too little context = fragile systems "built on sand with sticks."
- **Context is the cure.** The smartest AI people aren't building tools — they're building **context** (a "second brain").
- **Guardrails & lanes.** Give each agent a narrow job. Overload an agent with 10–12+ skills and they compete and fail.
- **Protect your wallet.** Never use auto-recharge. Agents can loop overnight and hand you a $10k bill.
- **Finish before you start more.** One project to "done" beats fifteen at 70%.

01 The problem, named

Two words you'll hear constantly: **drift** and **sprawl**. They're cousins, and together they're why a system that felt magical in week 1 feels like a part-time job by week 6.

> *"We've literally replaced maybe two full-time employees creating these workflows… the problem is the workflows break all the time. And I tried to spin up a manager to manage it…"*
>
> — Michael, on the 6/19 call

> *"AI can be lazy, take shortcuts, and create a fragile system built on sand with sticks that eventually requires a complete teardown... I spent the last 60 days rebuilding."*
>
> — from the room (Chris's experience)

🌀 Drift

The agent slowly stops behaving the way you set it up. Outputs wander off-voice, it "forgets" rules, it improvises. Caused by weak/contradictory context and too much loaded at once.

🧱 Sprawl

Too many things, half-finished: 15 workflows, 30 skills, 5 agents, memory files growing forever, cron jobs nobody remembers creating. Each new add makes the whole thing more fragile.

The trap (especially for non-coders): you keep *adding* to fix problems that adding *caused*. The fix is almost always **subtract + organize**, not add.

02 Diagnose it yourself (10-minute self-check)

Before you rebuild anything, score your setup. Answer yes/no honestly.

RED FLAGS

- Agent loaded with **10+ skills** (they start competing)
- No single **context file** (no `MEMORY.md` / `AGENTS.md`)
- Memory/notes files that **only grow**, never get pruned or archived
- Workflows you can't explain or that fail silently
- **Auto-recharge ON** for any AI account
- One mega-agent doing everything (no lanes)
- Heavy **Playwright/browser** automation where an API would do

GREEN FLAGS

- Each agent has a **narrow persona + 3–7 skills**
- A living **context store** ("second brain") it reads first
- Every folder has an **index** / table of contents
- Hard **spend limits**, no auto-recharge
- You can name what each workflow does in one sentence
- You **finish** projects before starting new ones

Run these checks on your own machine

For OpenClaw on a Mac mini — quick health/inventory commands. (Don't panic if a number is big; the point is to see the sprawl.)

```
openclaw status                  # is the gateway healthy?
openclaw cron list               # every scheduled job — kill what you don't r
ls ~/.openclaw/skills            # how many skills are loaded? (aim < ~8 per a
wc -l ~/.openclaw/workspace/MEMORY.md   # is your memory file ballooning?
openclaw tasks                   # background work still running / stuck loops
```

If `cron list` shows jobs you can't explain, or MEMORY.md is thousands of lines — that's your drift, in writing.

03 The fix — build a foundation (in order)

Do these in sequence. Each one makes the next easier. This is the same path that took a fragile setup to one that survives.

1. **Build your "second brain" first.** Before more agents, create an organized context store of *you* — your voice, brand, business, decisions. Feed it email, messages, calendar, key docs. This is what makes output come back "90% the way you want it." Tool of choice in the room: Obsidian. (See definition below.)

2. **Index everything.** The catch: more context = more tokens per question. So put a short index / table-of-contents at the top of every folder and big file so the agent can find the right slice instead of reading everything. Cheap, huge payoff.

3. **Write one context file.** Give the agent a single source of truth it reads first — in OpenClaw that's your `MEMORY.md` + `AGENTS.md` (Claude calls it `CLAUDE.md`; the generic term is "an MD file"). Rules, identity, do/don't, routing. This alone kills most drift.

4. **Give every agent a lane.** Treat agents like employees with job descriptions. Narrow persona, limited context, 3–7 skills max. A good agent will literally refuse work "outside its lane." That's a feature, not a bug.

5. **Prefer API over browser automation.** The single biggest cost lesson from the room: a skill built with Playwright (a robot clicking a browser) cost **$5,000**; rebuilt as a Python + API call it cost **36 cents**. If a real API exists, use it.

6 **Finish, then test meticulously.** One project to done. Interrogate the agent: "why did you choose that? is there a more reliable or secure way?" Don't run 15 half-built automations.

7 **Cap spend. No auto-recharge. Ever.** Set a hard limit. Agents get stuck in loops; people wake up to four-figure bills. This is non-negotiable.

04 Find your level

🟦 Beginner = "chat mode" 🟩 Intermediate = workflows 🟧 Advanced = agent fleet

Beginner

You use AI to search, draft, research — command by command. **You're already ahead of ~90% of people.** Next move: start a context file. One page about your business, your voice, your rules. That's it.

Intermediate

You've built workflows/automations (this is where Michael is). Pain = things break. Next move: stop adding, build the second brain + indexes, give each workflow one clear job, switch browser hacks to APIs.

Advanced

You run a fleet of specialized agents with personas + lanes. Next move: future-proof cost with **local LLMs** (run open models on your own hardware/GPU so token bills stop). Powerful, but more to manage.

05 Plain-English dictionary

Agent	An AI given a job, tools, and the ability to act on its own (not just chat back). Think "digital employee."
Drift	When an agent gradually stops following your setup — wanders off-voice, ignores rules. Usually a context problem.

Sprawl	Too many half-built parts (workflows, skills, agents, files) making the whole system fragile.
Context / Second brain	The organized knowledge you give an agent about you and your business so its output matches your intent. The real moat.
Skill	A specific capability you add to an agent (send email, read calendar). Too many at once = they compete and misfire.
Guardrails	Limits that keep an agent in its lane — narrow task, restricted tools, what it must never do.
Tokens	The "fuel" you pay for per word in/out. More context loaded = more tokens per question. Indexes reduce this.
Frontier model	The big paid cloud models (ChatGPT, Claude, etc.). Powerful, token-metered.
Local LLM	An open AI model running on your own hardware. No per-token bill — you just pay for power/compute.
MD file	A plain-text Markdown file (`.md`) used as an agent's instructions/context. OpenClaw: `MEMORY.md`, `AGENTS.md`. Claude: `CLAUDE.md`.
Playwright	A tool that drives a real web browser by clicking. Flexible but slow + expensive. Prefer an API when one exists.
API	A direct, structured way for software to talk to a service — far cheaper/faster than clicking through a browser.
Cron job	A scheduled task that runs on its own (e.g., every morning). Great — until you have forgotten ones running wild.

06 Tools mentioned in the room

OpenClaw
The agent runtime several of us run (often on a Mac mini). — docs.openclaw.ai

Obsidian
Markdown knowledge base — the go-to for building a "second brain." — obsidian.md

Claude Code / Codex
Coding agents — treat each like a specialist with a lane. — claude.com

Whisper
Turns voice/recordings into text to feed your second brain. — openai/whisper

Hugging Face
Open models + rentable GPUs ($0.50–$20/hr) to run local LLMs. — huggingface.co

GoHighLevel
The "I can do this for $279/mo instead of $18k" CRM example. — gohighlevel.com

07 Michael — your specific next 3 moves

You said it yourself on the call: agents on the Mac mini replacing ~2 FTEs, but workflows break, and you spun up a manager to babysit them. Here's the order I'd attack it:

1. **Don't add the "manager." Add context.** A manager-agent on top of fragile workflows just adds sprawl. First build one `MEMORY.md` /context file the agents read first. You said you "haven't done a ton of context work" — that's the whole ballgame.

2. **Audit the breakage.** List every workflow. For each: one sentence of what it does + is it API or browser? Kill or rebuild the browser-driven ones as API/Python (your own $5,000 → 36¢ lesson). Cut any agent carrying 10+ skills down to a lane.

3 **Cap spend + index your folders.** Turn OFF auto-recharge today. Add a short index to the top of each context folder so you're not burning tokens loading everything every time.

Do these three before building anything new. Foundation first, fleet second.

⚠ **Do this tonight:** Check every AI account and **turn off auto-recharge / set a hard spend cap.** A looping agent overnight is a real four-figure bill — multiple people in the room have the scars.

Nerds Mastermind — keep it simple, finish what you start, build context not chaos.
Grounded in the 6/19 session ("Leveraging AI Agents for Business Automation"). Tools/links verified against current docs. Questions → drop them in the thread.

Made for the crew · share freely.